

ALBERI RED-BLACK

UN ALBERO RB È UN ABR CON DUE CAMPI AGGIUNTIVI IN MEMORIA: IL COLORE DEL NODO, CHE PUÒ ESSERE RED O BLACK, E IL PUNTAIORE AL PADRE (PARENT). DEFINENDO DEI VINCOLI SU COME I NODI POSSONO ESSERE COLORATI, SI GARANTISCE CHE L'ALTEZZA DELL'ALBERO SIA LOGARITMICA RISPETTO AL NUMERO DEI NODI

IL CAMPO COLOR È RAPPRESENTATO CON UNA FUNZIONE DI TIPO BOOLEANA CHE ASSOCIA AD OGNI NODO UN BIT DA 0 A 1, A SECONDA CHE IL COLORE SIA ROSSO O NERO. IN QUESTO MODO, IL COLORE È RAPPRESENTATO CON UN SINGOLO BIT DI MEMORIA A DIFFERENZA DEL CAMPO ALTEZZA DEGLI AVL CHE È RAPPRESENTATO DA UN VALORE INTERO

key	
color	
p	
left	right

DEFINIZIONE

UN ALBERO RB È UN ABR CON LE SEGUENTI PROPRIETÀ:

1. OGNI NODO È COLORATO DI ROSSO O DI NERO
2. OGNI FOGLIA È COLORATA DI NERO E NON CONTIENE DATI, QUESTE FOGLIE SONO DETTE ESTERNE O NIL
3. LA RADICE È NERA
4. OGNI NODO ROSSO HA SOLO FIGLI NERI
5. PER OGNI NODO x , OGNI PERCORSO DA x A UN NODO NIL CONTIENE LO STESSO NUMERO DI NODI NERI. QUESTA QUANTITÀ È DETTA ALTEZZA NERA DI x E VIENE INDICATA CON $bh(x)$

ALTEZZA NERA

L'ALTEZZA NERA DI x , INDICATA CON $h_b(x)$ È IL NUMERO DI NODI NERI LUNGO UN CAMMINO SEMPLICE CHE INIZIA DAL NODO x (MA NON LO INCLUDE!) E TERMINA IN UNA FOGLIA.

PER DEFINIZIONE, L'ALTEZZA NERA DI UN ALBERO RED BLACK È L'ALTEZZA NERA DELLA SUA RADICE

LEMMA ALTEZZA MASSIMA RB

L'ALTEZZA MASSIMA DI UN ALBERO RED BLACK DI n NODI È $2 \log(n+1)$, CIOÈ

$$h \leq 2 \log(n+1)$$

RELAZIONE CAMMINO PIÙ LUNGO - CAMMINO PIÙ CORTO

IL CAMMINO PIÙ LUNGO DALLA RADICE A UNA FOGLIA (ALTERNATO ROSSO-NERO-ROSSO-NERO...) NON SARÀ MAI PIÙ LUNGO DEL DOPPIO DEL CAMMINO PIÙ CORTO (TUTTO NERO)

$$h \leq 2 h_b(x)$$

INSERIMENTO IN UN ALBERO RED BLACK

L'ALGORITMO DI INSERIMENTO IN UN ALBERO RB È MOLTO SIMILE A QUELLO PER GLI AVL ED HA 2 FASI:

1. INSERIMENTO STANDARD SECONDO LE REGOLE DI UN ABR
2. BILANCIAMENTO DELL'ALBERO, CON EVENTUALI RICOLOAZIONI

INSERT-RB(T, k) $O(\log n)$

IF $T == \text{NIL}$

RETURN NEW-NODE-RB(k)

IF $k > T \rightarrow \text{KEY}$

$T \rightarrow \text{RIGHT} = \text{INSERT-RB}(T \rightarrow \text{RIGHT}, k)$

$T = \text{BILANCIA-DX-RB}(T)$

ELSE IF $k < T \rightarrow \text{KEY}$

$T \rightarrow \text{LEFT} = \text{INSERT-RB}(T \rightarrow \text{LEFT}, k)$

$T = \text{BILANCIA-SX-RB}(T)$

return T

NEW-NODE-RB(k)

$\text{NODE} = \text{ALLOCATE-NODE}()$

$\text{NODE} \rightarrow \text{KEY} = k$

$\text{NODE} \rightarrow \text{LEFT} = \text{NIL}$

$\text{NODE} \rightarrow \text{RIGHT} = \text{NIL}$

$\text{NODE} \rightarrow \text{color} = \text{RED}$

RETURN NODE

BILANCIAMENTO IN UN ALBERO RB

IL BILANCIAMENTO SCATTA SOLO QUANDO SI VERIFICA UNA VIOLAZIONE DEL VINCOLO 4, OUVERO ALLA PRESENZA DI DUE NODI ROSSI CONSECUTIVI.

DATO CHE LA PROPRIETÀ 3 IMPONE IL COLORE NERO ALLA RADICE DELL'ALBERO, POSSIAMO GARANTIRE CHE OGNI ALBERO CON $h < 3$ NON NECESSITERÀ MAI DI RIBILANCIAMENTI:

- ALTEZZA 1: SOLO RADICE, NESSUN NODO ROSSO
- ALTEZZA 2: RADICE NERA, I DUE FIGLI NASCONO ROSSI
- ALTEZZA 3: UN NODO ROSSO AL LIVELLO 2 POTREBBE AVERE FIGLI ROSSI

POSSIAMO DIVIDERE I BILANCIAMENTI IN 3 CASI DISTINTI, TUTTI BASATI SULLO ZIO DEL NODO CHE SCATENA LO SBILANCIAMENTO

1. ZIO ROSSO: DATA LA CONDIZIONE DI SBILANCIAMENTO, PADRE, FIGLIO E ZIO SONO TUTTI ROSSI, ATTRAVERSO UNA SEMPLICE RICOLORAZIONE (PADRE E ZIO DIVENTANO NERI, IL NONNO DIVENTA ROSSO)

ATTENZIONE: LO SBILANCIAMENTO POTREBBE ESSERSI SPOSTATO, BISOGLIA RISALIRE L'ALBERO E CONTROLLARE

2. ZIO NERO - ZIG ZIG: SI EFFETTUA UNA ROTAZIONE SINISTRA SUL PADRE, SISTEMANDO LO ZIG-ZAG E APPLICANDO IL CASO 3

3. ZIO NERO - ZIG ZAG: COME PER GLI AVL, SI CHIEDE LA DOPPIA ROTAZIONE E SI SCAMBIANO I COLORI TRA LA VECCHIA RADICE DEL SOTTOALBERO E LA NUOVA

BILANCIA-SX-RB(T)

IF $T \rightarrow \text{LEFT} \neq \text{NIL}$

$L = T \rightarrow \text{LEFT}$

$R = T \rightarrow \text{RIGHT}$

IF HAS-RED-CONFLICT(L)

IF $R \neq \text{NIL} \ \&\& \ R \rightarrow \text{COLOR} == \text{RED}$

$L \rightarrow \text{COLOR} = \text{BLACK}$

$R \rightarrow \text{COLOR} = \text{BLACK}$

$T \rightarrow \text{COLOR} = \text{RED}$

ELSE

IF $L \rightarrow \text{RIGHT} \neq \text{NIL}$ AND $L \rightarrow \text{RIGHT} \rightarrow \text{COLOR} == \text{RED}$

$T \rightarrow \text{LEFT} = \text{ROTATIONE-SX}(L)$

$L = T \rightarrow \text{LEFT}$

$T = \text{ROTATIONE-DX}(T)$

$T \rightarrow \text{COLOR} = \text{BLACK}$

$T \rightarrow \text{RIGHT} \rightarrow \text{COLOR} = \text{RED}$

RETURN T

BILANCIA-SX-RB(T)

IF $T \rightarrow \text{LEFT} == \text{NIL}$

RETURN T

PARENT = $T \rightarrow \text{LEFT}$

UNCLE = $T \rightarrow \text{RIGHT}$

IF NOT HAS-RED-CONFLICT(PARENT)

RETURN T

IF IS-RED(UNCLE)

PARENT $\rightarrow$ COLOR = BLACK

UNCLE $\rightarrow$ COLOR = BLACK

$T \rightarrow$ COLOR = RED

RETURN T

CASO 1

IF IS-RED(PARENT $\rightarrow$ RIGHT)

$T \rightarrow \text{LEFT} = \text{ROTATIONE-SX-RB(PARENT)}$

PARENT = $T \rightarrow \text{LEFT}$

CASO 2

$T = \text{ROTATIONE-DX-RB(T)}$

$T \rightarrow$ COLOR = BLACK

$T \rightarrow \text{RIGHT} \rightarrow$ COLOR = RED

CASO 3

RETURN T

HAS-RED-CONFLICT(NODE)

RETURN IS-RED(NODE) && (IS-RED(NODE $\rightarrow$ LEFT) || IS-RED(NODE $\rightarrow$ RIGHT))

IS-RED(NODE)

RETURN NODE != NIL && NODE $\rightarrow$ COLOR == RED

BILANCIA-DX-RB(T)

IF $T \rightarrow \text{RIGHT} == \text{NIL}$

RETURN T

PARENT = $T \rightarrow \text{RIGHT}$

UNCLE = $T \rightarrow \text{LEFT}$

IF NOT HAS-RED-CONFLICT(PARENT)

RETURN T

IF IS-RED(UNCLE)

PARENT $\rightarrow$ COLOR = BLACK

UNCLE $\rightarrow$ COLOR = BLACK

T $\rightarrow$ COLOR = RED

RETURN T

IF IS-RED(PARENT $\rightarrow$ LEFT)

T $\rightarrow$ RIGHT = ROTAZIONE-DX-RB(PARENT)

PARENT = T $\rightarrow$ RIGHT

T = ROTAZIONE-SX-RB(T)

T $\rightarrow$ COLOR = BLACK

T $\rightarrow$ LEFT $\rightarrow$ COLOR = RED

RETURN T

ROTAZIONE-SX-RB(T)

NEWROOT = T $\rightarrow$ RIGHT

T $\rightarrow$ RIGHT = NEWROOT $\rightarrow$ LEFT

NEWROOT $\rightarrow$ LEFT = T

RETURN NEWROOT

ROTAZIONE-DX-RB(T)

NEWROOT = T $\rightarrow$ LEFT

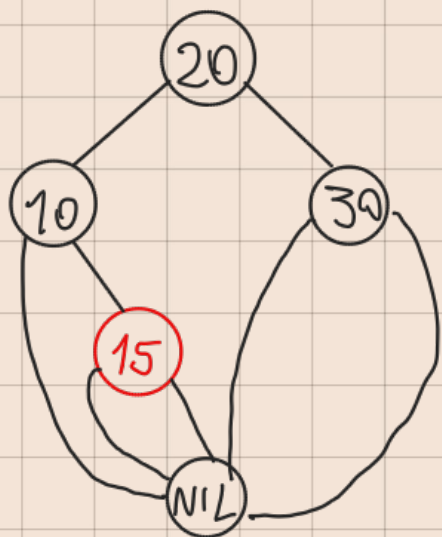
T $\rightarrow$ LEFT = NEWROOT $\rightarrow$ RIGHT

NEWROOT $\rightarrow$ RIGHT = T

RETURN NEWROOT

ESEMPIO DI INSERIMENTO

ALBERO INIZIALE:



EFFETTUAMO L'INSERIMENTO DELLA
CHIAVE [12]

INSERT-RB (T, k) // $T = [20], k = 12$

IF $T == \text{NIL}$

RETURN NEW-NODE-RB(k)

IF $k > T \rightarrow \text{KEY}$

$T \rightarrow \text{RIGHT} = \text{INSERT-RB}(T \rightarrow \text{RIGHT}, k)$

$T = \text{BILANCIA-DX-RB}(T)$

ELSE IF $k < T \rightarrow \text{KEY}$

• $T \rightarrow \text{LEFT} = \text{INSERT-RB}(T \rightarrow \text{LEFT}, k)$

$T = \text{BILANCIA-SX-RB}(T)$

return T

INSERT-RB2(T, k) // $T = [10]$, $k = 12$

IF $T == \text{NIL}$

RETURN NEW-NODE-RB(k)

IF $k > T \rightarrow \text{KEY}$

- $T \rightarrow \text{RIGHT} = \text{INSERT-RB}(T \rightarrow \text{RIGHT}, k)$

$T = \text{BILANCIA-DX-RB}(T)$

ELSE IF $k < T \rightarrow \text{KEY}$

$T \rightarrow \text{LEFT} = \text{INSERT-RB}(T \rightarrow \text{LEFT}, k)$

$T = \text{BILANCIA-SX-RB}(T)$

return T

INSERT-RB3(T, k) // $T = [15]$, $k = 12$

IF $T == \text{NIL}$

RETURN NEW-NODE-RB(k)

IF $k > T \rightarrow \text{KEY}$

$T \rightarrow \text{RIGHT} = \text{INSERT-RB}(T \rightarrow \text{RIGHT}, k)$

$T = \text{BILANCIA-DX-RB}(T)$

ELSE IF $k < T \rightarrow \text{KEY}$

- $T \rightarrow \text{LEFT} = \text{INSERT-RB}(T \rightarrow \text{LEFT}, k)$

$T = \text{BILANCIA-SX-RB}(T)$

return T

INSERT-RB4(T, k) // T = [NIL], k = 12

IF T == NIL

• RETURN NEW-NODE-RB(k) // RETURN [12, RED, NIL, NIL]

IF k > T->key

T->RIGHT = INSERT-RB(T->RIGHT, k)

T = BILANCIA-DX-RB(T)

ELSE IF k < T->key

T->LEFT = INSERT-RB(T->LEFT, k)

T = BILANCIA-SX-RB(T)

return T

NEW-NODE-RB(k) // k = 12

NODE = ALLOCATE-NODE() // NODE = []

NODE->key = k // NODE->key = 12

NODE->LEFT = NIL

NODE->RIGHT = NIL

NODE->color = RED

RETURN NODE // RETURN [12, RED, NIL, NIL]

INSERT-RB3(T, k) // T = [15], k = 12

IF T == NIL

RETURN NEW-NODE-RB(k)

IF k > T->key

T->RIGHT = INSERT-RB(T->RIGHT, k)

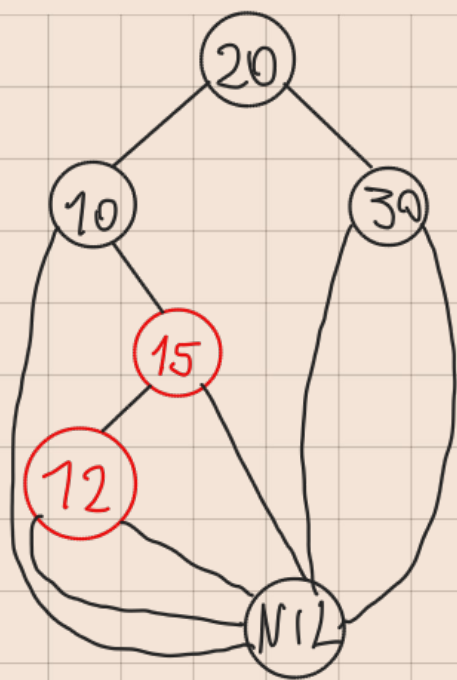
T = BILANCIA-DX-RB(T)

ELSE IF k < T->key

• T->LEFT = INSERT-RB(T->LEFT, k) [15] -> LEFT = [12, RED, NIL, NIL]

T = BILANCIA-SX-RB(T)

return T



INSERT-RB3(T, k) // $T = [15]$, $k = 12$

IF $T == \text{NIL}$

RETURN NEW-NODE-RB(k)

IF $k > T \rightarrow \text{key}$

$T \rightarrow \text{RIGHT} = \text{INSERT-RB}(T \rightarrow \text{RIGHT}, k)$

$T = \text{BILANCIA-DX-RB}(T)$

ELSE IF $k < T \rightarrow \text{key}$

$T \rightarrow \text{LEFT} = \text{INSERT-RB}(T \rightarrow \text{LEFT}, k)$

• $T = \text{BILANCIA-SX-RB}(T)$ // $\text{BILANCIA-SX-RB}([15])$

return T

BILANCIA-SX-RB(T) // T = [15]

IF T → LEFT == NIL

RETURN T

PARENT = T → LEFT // PARENT = [12]

UNCLE = T → RIGHT // UNCLE = NIL

IF NOT HAS-RED-CONFLICT(PARENT)

- RETURN T // RETURN [15]

...

...

HAS-RED-CONFLICT(NODE)

RETURN IS-RED(NODE) && (IS-RED(NODE → LEFT) || IS-RED(NODE → RIGHT)) // RETURN F

IS-RED(NODE)

RETURN NODE != NIL && NODE → color == RED

INSERT-RB(T, k) // T = [15], k = 12

IF T == NIL

RETURN NEW-NODE-RB(k)

IF k > T → key

T → RIGHT = INSERT-RB(T → RIGHT, k)

T = BILANCIA-SX-RB(T)

ELSE IF k < T → key

T → LEFT = INSERT-RB(T → LEFT, k)

T = BILANCIA-SX-RB(T) // T = [15]

- return T // RETURN [15]

INSERT-RB2(T, k) // T = [10], k = 12

IF T == NIL

RETURN NEW-NODE-RB(k)

IF k > T->key

T->RIGHT = INSERT-RB(T->RIGHT, k) // T->RIGHT = [15]

• T = BILANCIA-DX-RB(T) // T = BILANCIA-DX-RB([10])

ELSE IF k < T->key

T->LEFT = INSERT-RB(T->LEFT, k)

T = BILANCIA-SX-RB(T)

return T

BILANCIA-DX-RB(T) // T = [10]

IF T->RIGHT == NIL

RETURN T

PARENT = T->RIGHT // PARENT = [15]

UNCLE = T->LEFT // UNCLE = NIL

IF NOT HAS-RED-CONFLICT(PARENT)

RETURN T

IF IS-RED(UNCLE)

PARENT->COLOR = BLACK

UNCLE->COLOR = BLACK

T->COLOR = RED

RETURN T

IF IS-RED(PARENT->LEFT)

• T->RIGHT = ROTAZIONE-DX-RB(PARENT) // RDX([15])

PARENT = T->RIGHT

T = ROTAZIONE-SX-RB(T)

T->COLOR = BLACK

T->LEFT->COLOR = RED

RETURN T

HAS-RED-CONFLICT (NODE)

RETURN $\underset{T}{\text{IS-RED}(\text{NODE})} \&\& (\text{IS-RED}(\text{NODE} \rightarrow \underset{T}{\text{LEFT}}) \vee \text{IS-RED}(\text{NODE} \rightarrow \text{RIGHT}))$

IS-RED (NODE)

RETURN $\text{NODE} \neq \text{NIL} \&\& \text{NODE} \rightarrow \text{COLOR} \neq \text{RED}$

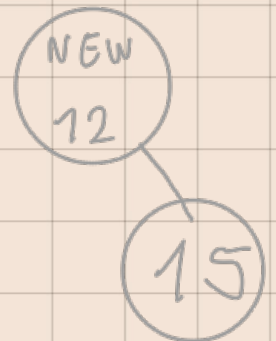
ROTAZIONE-DX-RB(T) // T = [15]

NEWROOT = T → LEFT // NEWROOT = [12]

T → LEFT = NEWROOT → RIGHT // [15] → LEFT = NIL

NEWROOT → RIGHT = T // [12] → RIGHT = [15]

RETURN NEWROOT // RETURN [12]



BILANCIA-DX-RB(T) // T = [10]

IF T → RIGHT == NIL

RETURN T

PARENT = T → RIGHT // PARENT = [15]

UNCLE = T → LEFT // UNCLE = NIL

IF NOT HAS-RED-CONFLICT(PARENT)

RETURN T

IF IS-RED(UNCLE)

PARENT → COLOR = BLACK

UNCLE → COLOR = BLACK

T → COLOR = RED

RETURN T

IF IS-RED(PARENT → LEFT)

T → RIGHT = ROTAZIONE-DX-RB(PARENT) // [10] → R = [12]

PARENT = T → RIGHT // PARENT = [12]

• T = ROTAZIONE-SX-RB(T) // T = RSX([10])

T → COLOR = BLACK

T → LEFT → COLOR = RED

RETURN T



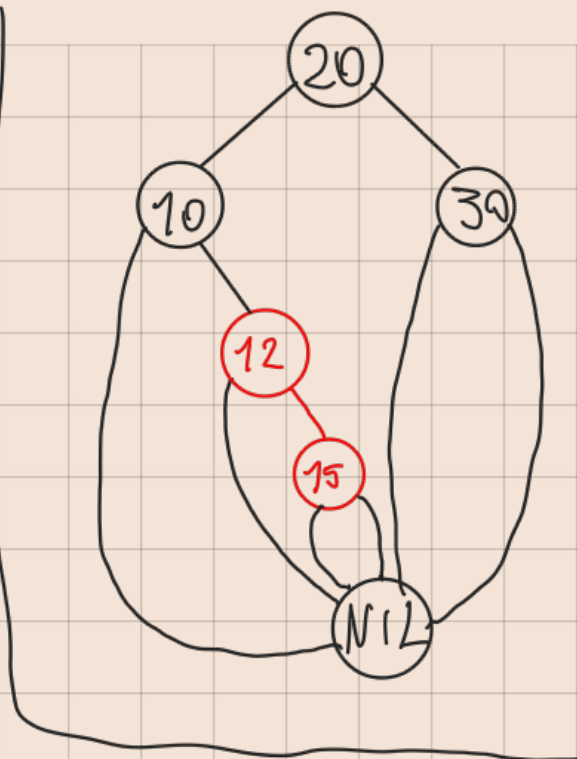
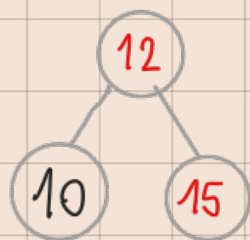
ROTATIONE-Sx-RB(T) // T = [10]

NEWROOT = T → RIGHT // NEW = [12]

T → RIGHT = NEWROOT → LEFT // 10 → R = NIL

NEWROOT → LEFT = T // NEW → L = [10]

RETURN NEWROOT // RET. [12]



BILANCIA-Dx-RB(T) // T = [10]

IF T → RIGHT == NIL

RETURN T

PARENT = T → RIGHT // PARENT = [15]

UNCLE = T → LEFT // UNCLE = NIL

IF NOT HAS-RED-CONFLICT(PARENT)

RETURN T

IF IS-RED(UNCLE)

PARENT → COLOR = BLACK

UNCLE → COLOR = BLACK

T → COLOR = RED

RETURN T

IF IS-RED(PARENT → LEFT)

T → RIGHT = ROTATIONE-Dx-RB(PARENT) // [10] → R = [12]

PARENT = T → RIGHT // PARENT = [12]

T = ROTATIONE-Sx-RB(T) // T = [12]

T → COLOR = BLACK // [12] → COLOR = BLACK

T → LEFT → COLOR = RED // [10] → COLOR = RED

RETURN T // RETURN [12]

INSERT-RB2(T, k) // T = [10], k = 12

IF T == NIL

RETURN NEW-NODE-RB(k)

IF k > T->key

T->RIGHT = INSERT-RB(T->RIGHT, k) // T->RIGHT = [15]

T = BALANCE-DX-RB(T) // T = [12]

ELSE IF k < T->key

T->LEFT = INSERT-RB(T->LEFT, k)

T = BALANCE-SX-RB(T)

return T // RETURN [12]

INSERT-RB(T, k) // T = [20], k = 12

IF T == NIL

RETURN NEW-NODE-RB(k)

IF k > T->key

T->RIGHT = INSERT-RB(T->RIGHT, k)

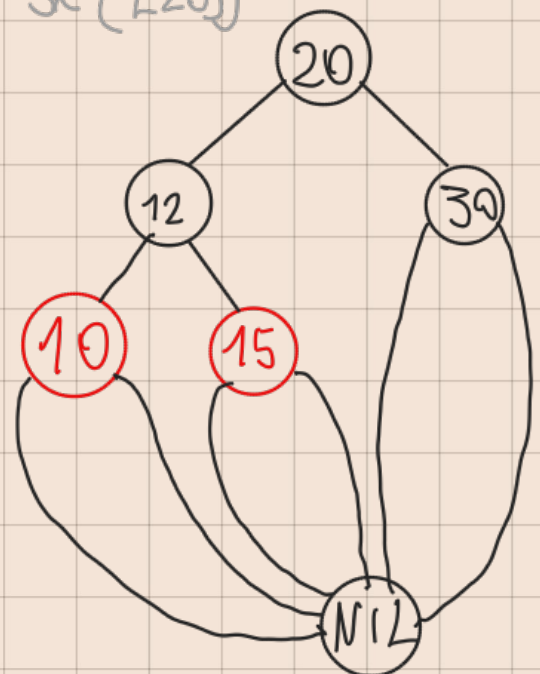
T = BALANCE-DX-RB(T)

ELSE IF k < T->key

T->LEFT = INSERT-RB(T->LEFT, k) // [20]->LEFT = [12]

* T = BALANCE-SX-RB(T) // BAL-SX([20])

return T



BILANCIA-SX-RB(T) // T = [20]

IF T → LEFT == NIL

RETURN T

PARENT = T → LEFT // PARENT = [12]

UNCLE = T → RIGHT // UNCLE = [30]

IF NOT HAS-RED-CONFLICT(PARENT)

RETURN T // RETURN [20]

...

INSERT-RB(T, k) // T = [20], k = 12

IF T == NIL

RETURN NEW-NODE-RB(k)

IF k > T → KEY

T → RIGHT = INSERT-RB(T → RIGHT, k)

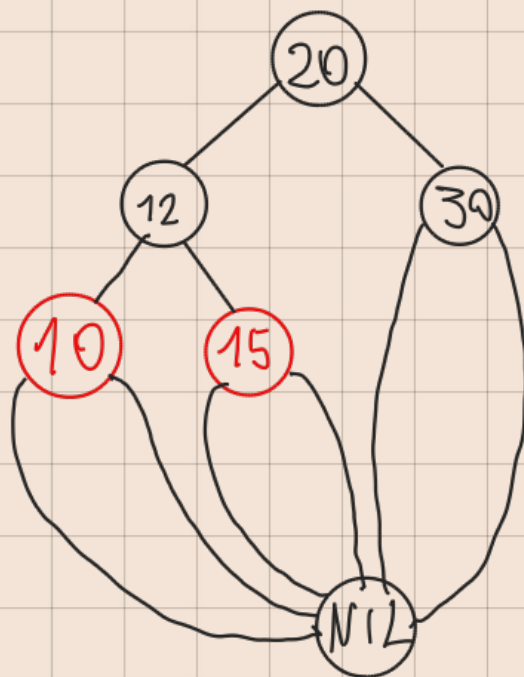
T = BILANCIA-SX-RB(T)

ELSE IF k < T → KEY

T → LEFT = INSERT-RB(T → LEFT, k) // [20] → LEFT = [12]

T = BILANCIA-SX-RB(T) // T = [20]

return T // RETURN 20



CANCELLAZIONE IN ALBERO RED-BLACK

A DIFFERENZA DELL'OPERAZIONE DI INSERIMENTO, IN CUI IL PERICOLO DI SBILANCIAMENTO ERA DATO DALLA PRESENZA DI NODI ROSSI CONSECUTIVI (VIOLAZIONE DELLA PROPRIETÀ₁), NELLA CANCELLAZIONE IL PROBLEMA È DATO DALLA VARIAZIONE DELL'ALTEZZA NERA (VIOLAZIONE DELLA PROPRIETÀ₅). DURANTE LA RIMOZIONE DI UN NODO SI PRESENTANO 2 SCENARI:

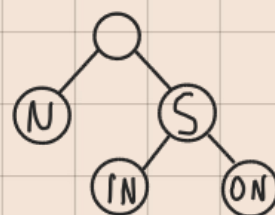
1. CANCELLAZIONE DI UN NODO ROSSO: TUTTI I CAMMINI CHE PASSAVANO PER QUEL NODO MANTENGONO LA STESSA ALTEZZA NERA, NON È NECESSARIO RIBILANCIARE
2. CANCELLAZIONE DI UN NODO NERO: TUTTI I CAMMINI PASSANTI PER QUEL NODO SUBISCONO UNA RIDUZIONE DELL'ALTEZZA NERA, INVALIDANDO L'INTERA STRUTTURA

PER IL RIPRISTINO DELL'EQUILIBRIO, L'ALGORITMO UTILIZZA UN ESPEDIENTE LOGICO: ADDOSSA IL COLORE DEL NODO ELIMINATO A QUELLO CHE PRENDE IL SUO POSTO, GENERANDO 2 ESITI:

- IL SOSTITUTO È ROSSO: ASSORBE IL NERO ELIMINATO, DIVENTANDO UN NODO NERO, CONCLUDENDO IL BILANCIAMENTO
- IL SOSTITUTO È NERO: ASSORBENDO UN ULTERIORE LIVELLO DI NERO, IL NODO ASSUME UNO STATO LOGICO TEMPORANEO, CHIAMATO DOPPIO NERO (DOUBLE BLACK, DB)

PER DECIDERE COME SCARICARE IL DOPPIO NERO DA N, L'ALGORITMO NON GUARDA PIÙ LO ZIO (COME NELL'INSERIMENTO), MA IL LATO OPPOSTO DELL'ALBERO. DEFINIAMO GLI ATTORI:

- N (NODE): IL NODO IN ESAME, IL DOPPIO NERO
- S (SIBLING): IL FRATELLO DI N
- IN (INNER NEPHEW): IL FIGLIO DI S PIÙ "VICINO" AD N
- ON (OUTER NEPHEW): IL FIGLIO DI S PIÙ "LONTANO" DA N



14 CASI DI BILANCIAMENTO

L'ALGORITMO ANALIZZA LO STATO DI S E DEI SUOI FIGLI:

1. S È ROSSO: SE S È ROSSO, IL PADRE DI S È NERO. VIENE COLORATO S DI NERO E IL PADRE DI ROSSO, SI APPLICA POI UNA ROTAZIONE SUL PADRE, IN DIREZIONE DI N, N È DOPPIO NERO E SI RICADE IN UNO DEI CASI 2, 3, 4
2. S È NERO, IN E ON SONO NERI: IL FRATELLO NON HA LA POSSIBILITÀ DI SACRIFICARE UN PESO ROSSO PER ASSORBIRE L'ANOMALIA. VIENE ESTRATTO UN PESO NERO SIA DA N (CHE TORNA NERO) CHE DA S, CHE DIVENTA ROSSO, IL PESO NERO SALE AL PADRE. SE IL PADRE ERA ROSSO, DIVENTA NERO E IL BILANCIAMENTO TERMINA. SE IL PADRE ERA NERO, DIVENTA DOPPIO NERO E IL CICLO RICOMINCIA
3. S È NERO, IN È ROSSO E ON È NERO: C'È UN DIBALANCIAMENTO GEOMETRICO DEI NODI ROSSI. SI RICOLORA IN DI NERO E S DI ROSSO, POI SI ESEGUE UNA ROTAZIONE SU S PER ALLONTANARLO DA N. I NODI SI RITROVANO INVERTITI, CON IN CHE È DIVENTATO FRATELLO DI N. RICADIAMO NEL CASO 4.
4. S È NERO, ON È ROSSO: S ASSORBE IL COLORE DEL PADRE, IL PADRE DIVENTA NERO INSIEME AD ON. INFINE SI ESEGUE UNA ROTAZIONE SUL PADRE, FACENDOLO "SCENDERE" VERSO N

DELETE-RB(T, k)

IF $T == \text{NIL}$

RETURN NIL

IF $k < T \rightarrow \text{KEY}$

$T \rightarrow \text{LEFT} = \text{DELETE-RB}(T \rightarrow \text{LEFT}, k)$

$T = \text{BILANCIA-DEL-SX-RB}(T)$

ELSE IF $k > T \rightarrow \text{KEY}$

$T \rightarrow \text{RIGHT} = \text{DELETE-RB}(T \rightarrow \text{RIGHT}, k)$

$T = \text{BILANCIA-DEL-DX-RB}(T)$

ELSE

$T = \text{DELETE-ROOT-RB}(T)$

RETURN T

DELETE-ROOT-RB(T)

IF $T \rightarrow \text{LEFT} == \text{NIL}$ // CASO 1: MANCA IL FIGLIO SINISTRO

$\text{NEWROOT} = T \rightarrow \text{RIGHT}$

IF $T \rightarrow \text{COLOR} == \text{BLACK}$

PROPAGATE-BLACK(NEWROOT)

FREE(T)

RETURN NEWROOT

IF $T \rightarrow \text{RIGHT} == \text{NIL}$ // CASO 2: MANCA IL FIGLIO DESTRO

$\text{NEWROOT} = T \rightarrow \text{LEFT}$

IF $T \rightarrow \text{COLOR} == \text{BLACK}$

PROPAGATE-BLACK(NEWROOT)

FREE(T)

RETURN NEWROOT

SUCCESSORE = MINIMO($T \rightarrow \text{RIGHT}$) // CASO 3: ENTRAMBI I FIGLI PRESENTI

$T \rightarrow \text{KEY} = \text{SUCCESSORE} \rightarrow \text{KEY}$

$T \rightarrow \text{RIGHT} = \text{DELETE-RB}(T \rightarrow \text{RIGHT}, \text{SUCCESSORE} \rightarrow \text{KEY})$

RETURN T

MINIMO

IF $T \rightarrow \text{LEFT} == \text{NIL}$

RETURN T

RETURN MINIMO ($T \rightarrow \text{LEFT}$)

PROPAGATE-BLACK(T)

IF $T \neq \text{NIL}$ & $T \rightarrow \text{COLOR} == \text{RED}$

$T \rightarrow \text{COLOR} = \text{BLACK}$

ELSE

$T \rightarrow \text{COLOR} = \text{DOUBLE BLACK}$

BILANCIA-DEL-SX-RB(T)

N = T → LEFT

IF NOT IS-DOUBLE-BLACK(N)

RETURN T

S = T → RIGHT

IF IS-RED(S)

S → COLOR = BLACK

T → COLOR = RED

T = ROTAZIONE-SX(T)

T → LEFT = BILANCIA-DEL-SX-RB(T → LEFT)

RETURN T

IN = S → LEFT

ON = S → RIGHT

IF IS-BLACK(IN) AND IS-BLACK(ON)

S → COLOR = RED

REMOVE-DOUBLE-BLACK(N)

PROPAGATE-BLACK(T)

RETURN T

IF IS-BLACK(ON)

IN → COLOR = BLACK

S → COLOR = RED

T → RIGHT = ROTAZIONE-DX(S)

S = T → RIGHT

ON = S → RIGHT

S → COLOR = T → COLOR

T → COLOR = BLACK

ON → COLOR = BLACK

REMOVE-DOUBLE-BLACK(N)

T = ROTAZIONE-SX(T)

RETURN T

FUNCTIONS TO SUPPORT

IS-DOUBLE-BLACK(NODE)

RETURN NODE != NIL & & NODE->COLOR == DOUBLE-BLACK

REMOVE-DOUBLE-BLACK(NODE)

IF NODE != NIL

NODE->COLOR = BLACK

RETURN

IS-BLACK(NODE)

RETURN !IS-RED(NODE)

BILANCIA-DEL-DX-RB (T)

N = T → RIGHT

IF NOT IS-DOUBLE-BLACK(N)

RETURN T

S = T → LEFT

IF IS-RED(S)

S → COLOR = BLACK

T → COLOR = RED

T = ROTAZIONE-DX(T)

T → RIGHT = BILANCIA-DEL-DX-RB(T → LEFT)

RETURN T

IN = S → RIGHT

ON = S → LEFT

IF IS-BLACK(IN) AND IS-BLACK(ON)

S → COLOR = RED

REMOVE-DOUBLE-BLACK(N)

PROPAGATE-BLACK(T)

RETURN T

IF IS-BLACK(ON)

IN → COLOR = BLACK

S → COLOR = RED

T → LEFT = ROTAZIONE-SX(S)

S = T → LEFT

ON = S → LEFT

S → COLOR = T → COLOR

T → COLOR = BLACK

ON → COLOR = BLACK

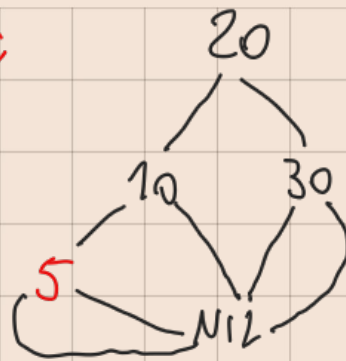
REMOVE-DOUBLE-BLACK(N)

T = ROTAZIONE-DX(T)

RETURN T

ESEMPIO DI ESECUZIONE

ALBERO INIZIALE:



DELETE-RB¹(T, k) // T = 20, k = 30

IF T == NIL

RETURN NIL

IF k < T → KEY

T → LEFT = DELETE-RB(T → LEFT, k)

T = BILANCIA-DEL-SX-RB(T)

ELSE IF k > T → KEY

• T → RIGHT = DELETE-RB(T → RIGHT, k) T → RIGHT = DRB(30, 30)

T = BILANCIA-DEL-DX-RB(T)

ELSE

T = DELETE-ROOT-RB(T)

RETURN T

DELETE-RB²(T, k) // T = 30, k = 30

IF T == NIL

RETURN NIL

IF k < T → KEY

T → LEFT = DELETE-RB(T → LEFT, k)

T = BILANCIA-DEL-SX-RB(T)

ELSE IF k > T → KEY

T → RIGHT = DELETE-RB(T → RIGHT, k)

T = BILANCIA-DEL-DX-RB(T)

ELSE

• T = DELETE-ROOT-RB(T) // T = DELETE-ROOT-RB(30)

RETURN T

DELETE-ROOT-RB(T) // T = 30

IF T → LEFT == NIL

NEWROOT = T → RIGHT // NEWROOT = NIL

IF T → COLOR == BLACK

• PROPAGATE-BLACK(NEWROOT) // PROPAGATE-BLACK(NIL)

FREE(T)

RETURN NEWROOT

PROPAGATE-BLACK(T) // T = NIL

IF T != NIL && T → COLOR == RED

T → COLOR = BLACK

ELSE

T → COLOR = DOUBLE BLACK // NIL → COLOR = DOUBLE-BLACK

DELETE-ROOT-RB(T) // T = 30

IF T → LEFT == NIL

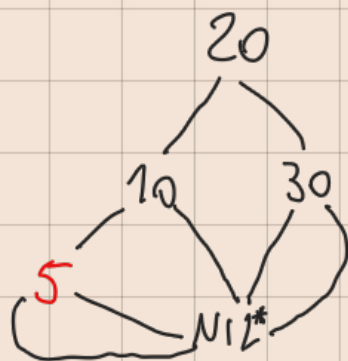
NEWROOT = T → RIGHT // NEWROOT = NIL

IF T → COLOR == BLACK

PROPAGATE-BLACK(NEWROOT) // PROPAGATE-BLACK(NIL)

FREE(T)

RETURN NEWROOT // RETURN NIL



* DOUBLE
BLACK

DELETE-RB²(T, k) // T = 30, k = 30

IF T == NIL

RETURN NIL

IF k < T → KEY

T → LEFT = DELETE-RB(T → LEFT, k)

T = BALANCE-DEL-SX-RB(T)

ELSE IF k > T → KEY

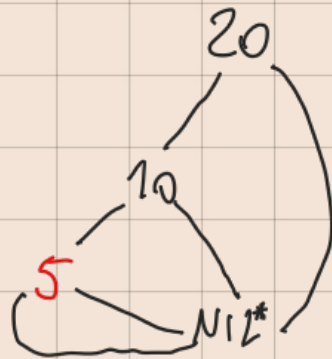
T → RIGHT = DELETE-RB(T → RIGHT, k)

T = BALANCE-DEL-DX-RB(T)

ELSE

T = DELETE-ROOT-RB(T) // T = NIL

RETURN T // RETURN NIL



* DOUBLE
BLACK

DELETE-RB¹(T, k) // T = 20, k = 30

IF T == NIL

RETURN NIL

IF $k \leq T \rightarrow \text{KEY}$

$T \rightarrow \text{LEFT} = \text{DELETE-RB}(T \rightarrow \text{LEFT}, k)$

$T = \text{BILANCIA-DEL-SX-RB}(T)$

ELSE IF $k > T \rightarrow \text{KEY}$

$T \rightarrow \text{RIGHT} = \text{DELETE-RB}(T \rightarrow \text{RIGHT}, k)$ $T \rightarrow \text{RIGHT} = \text{DRB}(30, 30)$

• $T = \text{BILANCIA-DEL-DX-RB}(T)$ // $T = \text{BILANCIA-DEL-DX-RB}(20)$

ELSE

$T = \text{DELETE-ROOT-RB}(T)$

RETURN T

BILANCIA-DEL-DX-RB(T) // T = 20

N = T → RIGHT // N = NIL*

IF NOT IS-DOUBLE-BLACK(N)

RETURN T

S = T → LEFT // S = 10

IF IS-RED(S)

S → COLOR = BLACK

T → COLOR = RED

T = ROTAZIONE-DX(T)

T → RIGHT = BILANCIA-DEL-DX-RB(T → LEFT)

RETURN T

IN = S → RIGHT // IN = NIL

ON = S → LEFT // ON = S

IF IS-BLACK(IN) AND IS-BLACK(ON) F

S → COLOR = RED

REMOVE-DOUBLE-BLACK(N)

PROPAGATE-BLACK(T)

RETURN T

IF IS-BLACK(ON) F

IN → COLOR = BLACK

S → COLOR = RED

T → LEFT = ROTAZIONE-SX(S)

S = T → LEFT

ON = S → LEFT

S → COLOR = T → COLOR // 10 → COLOR = BLACK

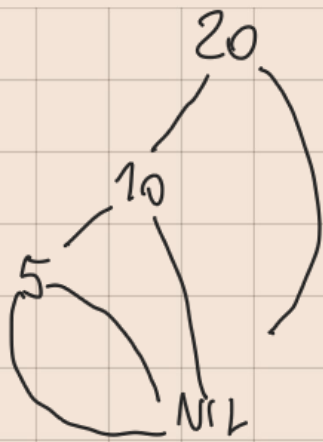
T → COLOR = BLACK // 20 → COLOR = BLACK

ON → COLOR = BLACK // 5 → COLOR = BLACK

REMOVE-DOUBLE-BLACK(N) NIL → COLOR = BLACK

T = ROTAZIONE-DX(T) // T = RDx(20)

RETURN T



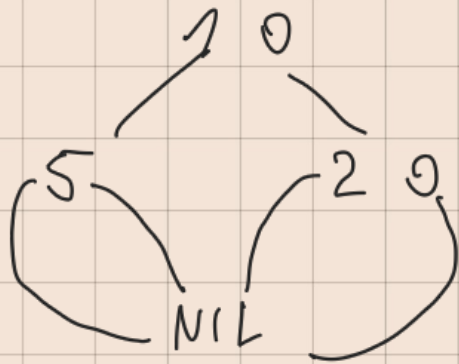
ROTATIONE-DX-RB(T) // T = 20

NEWROOT = T → LEFT // NEWROOT = 10

T → LEFT = NEWROOT → RIGHT // 20 → LEFT = NIL

NEWROOT → RIGHT = T // 10 → RIGHT = 20

RETURN NEWROOT // RETURN 10



BILANCIA-DEL-DX-RB(T) // T = 20

N = T → RIGHT // N = NIL*

IF NOT IS-DOUBLE-BLACK(N)

RETURN T

S = T → LEFT // S = 10

IF IS-RED(S)

S → COLOR = BLACK

T → COLOR = RED

T = ROTAZIONE-DX(T)

T → RIGHT = BILANCIA-DEL-DX-RB(T → LEFT)

RETURN T

IN = S → RIGHT // IN = NIL

ON = S → LEFT // ON = S

IF IS-BLACK(IN) AND IS-BLACK(ON) F

S → COLOR = RED

REMOVE-DOUBLE-BLACK(N)

PROPAGATE-BLACK(T)

RETURN T

IF IS-BLACK(ON) F

IN → COLOR = BLACK

S → COLOR = RED

T → LEFT = ROTAZIONE-SX(S)

S = T → LEFT

ON = S → LEFT

S → COLOR = T → COLOR // 10 → COLOR = BLACK

T → COLOR = BLACK // 20 → COLOR = BLACK

ON → COLOR = BLACK // 5 → COLOR = BLACK

REMOVE-DOUBLE-BLACK(N) NIL → COLOR = BLACK

T = ROTAZIONE-DX(T) // T = 10

RETURN T // RETURN 10